A. Supreeth Reddy

2303A51392

Batch 06

AIAC Lab 9.3

Task 1: Basic Docstring Generation
Scenario
You are developing a utility function that processes numerical lists and must be properly documented for future maintenance.
Requirements
• Write a Python function to return the sum of even numbers and sum of odd numbers in a given list
• Manually add a Google Style docstring to the function
• Use an AI-assisted tool (Copilot / Cursor AI) to generate a function-level docstring
• Compare the AI-generated docstring with the manually written docstring
• Analyze clarity, correctness, and completeness
Expected Output
• Python function with manual Google-style docstring
• AI-generated docstring for the same function
• Comparison explaining differences between manual and AI-generated documentation
• Improved understanding of AI-generated function-level documentation

#PROMPT: #write a python function to return sum of odd and sum of even nummbers from a list
#PROMPT2: #manually add a docstring to the above function

CODE:
```
def sum_odd_even(numbers):
    sum_odd = 0
    sum_even = 0
    for num in numbers:
        if num % 2 == 0:
            sum_even += num
        else:
            sum_odd += num
    return sum_odd, sum_even

def sum_odd_even(numbers):
    """
    This function takes a list of numbers as input and returns the sum of odd numbers and the sum of even numbers in the list.

    Parameters:
    numbers (list): A list of integers.
```

```
    Returns:
    tuple: A tuple containing the sum of odd numbers and the sum of even numbers.
    """
    sum_odd = 0
    sum_even = 0
    for num in numbers:
        if num % 2 == 0:
            sum_even += num
        else:
            sum_odd += num
    return sum_odd, sum_even
#example usage
numbers = [1, 2, 3, 4, 5, 6]
odd_sum, even_sum = sum_odd_even(numbers)
print(f"Sum of odd numbers: {odd_sum}")
print(f"Sum of even numbers: {even_sum}")
```

OUTPUT:

```
Sum of odd numbers: 9
Sum of even numbers: 12
```

ask 2: Automatic Inline Comments

Scenario

You are developing a student management module that must be easy to understand for new developers.

Requirements

• Write a Python program for an sru_student class with the following:

– Attributes: name, roll_no, hostel_status

– Methods: fee_update() and display_details()

• Manually write inline comments for each line or logical block

• Use an AI-assisted tool to automatically add inline comments

• Compare manual comments with AI-generated comments

• Identify missing, redundant, or incorrect AI comments

Expected Output

• Python class with manually written inline comments

• AI-generated inline comments added to the same code

• Comparative analysis of manual vs AI comments

• Critical discussion on strengths and limitations of AI-generated comments

#PROMPT: #write a class sru_stduent name,rollno,hostel_status and methods fee_update and display_details

CODE:

```
class SRUStudent:
    def __init__(self, name, rollno, hostel_status):
        self.name = name #name of the student
        self.rollno = rollno #roll number of the student
```

```
        self.hostel_status = hostel_status #hostel status of the student (e.g., "Yes" or "No")
        self.fee = 0 #initial fee is set to 0

    def fee_update(self, amount):#method to update the fee by adding the specified amount to the current fee
        self.fee += amount
        print(f"Fee updated. Current fee: {self.fee}")

    def display_details(self):#method to display the details of the student, including name, roll number, hostel status, and current fee
        print(f"Name: {self.name}")
        print(f"Roll No: {self.rollno}")
        print(f"Hostel Status: {self.hostel_status}")
        print(f"Current Fee: {self.fee}")

#example usage
student1 = SRUStudent("Alice", "SRU123", "Yes")
student1.display_details() # Display initial details
student1.fee_update(500) # Update fee by adding 500
student1.display_details() # Display updated details
```

OUTPUT:

```
Name: Alice
Roll No: SRU123
Hostel Status: Yes
Current Fee: 0
Fee updated. Current fee: 500
Name: Alice
Roll No: SRU123
Hostel Status: Yes
Current Fee: 500
```

Task 3: Module-Level and Function-Level Documentation

Scenario

You are building a small calculator module that will be shared across multiple projects and requires structured documentation.

Requirements

• Write a Python script containing 3–4 functions (e.g., add, subtract, multiply, divide)
• Manually write NumPy Style docstrings for each function
• Use AI assistance to generate:
– A module-level docstring
– Individual function-level docstrings
• Compare AI-generated docstrings with manually written ones
• Evaluate documentation structure, accuracy, and readability

Expected Output

• Python script with manual NumPy-style docstrings
• AI-generated module-level and function-level documentation

• Comparison between AI-generated and manual documentation
• Clear understanding of structured documentation for multi-function scripts
Additional Requirement
• Push the complete project documentation as a .md file to a GitHub repository
• Ensure documentation covers module overview and function descriptions

#PROMPT: #write a code for simple calculator using functions for addition, subtraction, multiplication, and division

#code:
```
def add(a, b): #addition function that takes two numbers as input and returns their sum
    return a + b
def subtract(a, b):#subtraction function that takes two numbers as input and returns the result of subtracting the second number from the first number
    return a - b
def multiply(a, b):#multiplication function that takes two numbers as input and returns their product
    return a * b
def divide(a, b):#division function that takes two numbers as input and returns the result of dividing the first number by the second number. It also checks for division by zero and returns an error message if the second number is zero.
    if b != 0:
        return a / b
    else:
        return "Error: Division by zero is not allowed."
#example usage
num1 = float(input("Enter the first number: "))#taking input from the user for the first number and converting it to a float
num2 = float(input("Enter the second number: "))#taking input from the user for the second number and converting it to a float
operation = input("Choose an operation (add, subtract, multiply, divide): ").lower()#taking input from the user for the desired operation and converting it to lowercase for easier comparison
if operation == "add":#calling the add function with the two numbers as arguments and storing the result in the variable 'result'
    result = add(num1, num2)
    print(f"The result of addition is: {result}")
elif operation == "subtract":
    result = subtract(num1, num2)#calling the subtract function with the two numbers as arguments and storing the result in the variable 'result'
    print(f"The result of subtraction is: {result}")
elif operation == "multiply":
    result = multiply(num1, num2)#calling the multiply function with the two numbers as arguments and storing the result in the variable 'result'
    print(f"The result of multiplication is: {result}")
elif operation == "divide":
```

```
    result = divide(num1, num2)#calling the divide function with the two numbers as
arguments and storing the result in the variable 'result'. The divide function will handle
division by zero and return an appropriate message if necessary.
    print(f"The result of division is: {result}")
else:
    print("Invalid operation selected.")#printing an error message if the user selects an invalid
operation that is not among the specified options (add, subtract, multiply, divide).
#example usage
num1=10
num2=5
print(f"Addition: {add(num1, num2)}")#calling the add function with num1 and num2 as
arguments and printing the result of addition
print(f"Subtraction: {subtract(num1, num2)}")#calling the subtract function with num1 and
num2 as arguments and printing the result of subtraction
print(f"Multiplication: {multiply(num1, num2)}")#calling the multiply function with num1
and num2 as arguments and printing the result of multiplication
print(f"Division: {divide(num1, num2)}")#calling the divide function with num1 and num2 as
arguments and printing the result of division. The divide function will handle division by zero
if num2 is zero and return an appropriate message.
```

OUTPUT:

```
Enter the first number: 10
Enter the second number: 5
Choose an operation (add, subtract, multiply, divide): add
The result of addition is: 15.0
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
```